Then, $\pi_1(\mu)$ is a measure on $(X + \bot, \Sigma_{X+\bot})$ and $\pi_2(\mu)$ is a measure on $(Y + \bot, \Sigma_{Y+\bot})$.

F.2 Measure Space on Expressions

To make the probabilistic semantics precise, we construct an appropriate measure space structure on the set of expressions by factoring out real numbers using the notion of what we call *expression skeletons*.

Definition F.7 (Expression Skeleton). An expression skeleton s is a syntactic expression obtained from a program by replacing each occurrence of a float constant with a hole of the form $\square$. The set of skeletons is generated by the grammar:

$$\begin{aligned} s ::= {} & \square \mid x \mid \text{true} \mid \text{false} \mid k_{\#n} \mid \textbf{diverge} \mid () \\ & \mid \textbf{let}\ x = s_1\ \textbf{in}\ s_2 \mid \textbf{if}\ s_1\ \textbf{then}\ s_2\ \textbf{else}\ s_3 \\ & \mid s_1 < s_2 \mid s_1 <_{\#n} s_2 \mid s_1 ==_{\#n} s_2 \\ & \mid \textbf{uniform}(s_1, s_2) \mid \textbf{discrete}(p_0, \ldots, p_{n-1}) \\ & \mid s_1\ \&\&\ s_2 \mid s_1\ ||\ s_2 \mid \text{not}\ s \mid \textbf{observe}(s) \\ & \mid (s_1, s_2) \mid \textbf{fst}\ s \mid \textbf{snd}\ s \\ & \mid \textbf{fun}\ x \to s \mid s_1\ s_2 \mid \textbf{fix}\ f\ x := s \\ & \mid \texttt{nil} \mid s_1 :: s_2 \\ & \mid (\textbf{match}\ s\ \textbf{with} \mid \texttt{nil} \to s_{\text{nil}} \mid h :: t \to s_{\text{cons}}\ \textbf{end}) \end{aligned}$$

In the discrete form, $n > 0$ and the probability literals p_i are fixed parts of the syntax, with $p_i \geq 0$ and $\sum_i p_i = 1$. They are not float holes: the measurable coordinates model program float constants, whereas a change to a discrete probability literal selects a different skeleton. We denote by *Skel* the set of all expression skeletons, and by . The holes of a skeleton are its occurrences of $\square$, ordered from left to right in a preorder traversal of the syntax tree; $holes(s)$ is the number of distinct holes in skeleton sthese occurrences. We write $\square_i$ for the ith occurrence when referring to a particular hole. Thus, the subscripts distinguish occurrences and are not part of the skeleton syntax.

Example F.8. The expression $\textbf{uniform}(0.0, 1.0) < 0.5$ has the skeleton $\textbf{uniform}(\square, \square) < \square$ with three holes.

Definition F.9 (Filling a skeleton). Let $s \in Skel$ and let $\vec{r} = (r_1, \ldots, r_{holes(s)}) \in \mathbb{R}^{holes(s)}$. Define the expression $s[\vec{r}] \in Expr$ to be the result of replacing each hole the ith hole occurrence $\square_i$ in s with the corresponding real constant r_i, for $1 \leq i \leq holes(s)$.

To recover a *typed* notion of skeletons, we extend the typing judgment to skeletons by adding a distinguished typing rule for holes.

Definition F.10 (Well-typed expression skeleton). Define a typing judgment for skeletons, written $\Gamma \vdash s : \tau$, with the understanding that all typing rules from expressions are preserved for skeletons, and that holes are typed as $\textbf{float}[B; V]$ according to the rule below:

$$\frac{}{\Gamma \vdash \square : \textbf{float}[B; V]}$$

We denote by $Skel_\tau$ the set of all well-typed expression skeletons, where:

$$Skel_\tau := \{s \in Skel \mid s : \tau\}.$$

Remark F.11. Both $Skel_\tau$ and $Expr_\tau$ are to be understood as the set of all well-typed skeletons and expressions, respectively, *equipped with their full subtype information.*

For example,

$$\square : \textbf{float}[B; \{0.5\}] < \square : \textbf{float}[B; \{0.5\}] \quad \text{and} \quad \square : \textbf{float}[B; \{0.2\}] < \square : \textbf{float}[B; \{0.8\}]$$

are distinct elements of $Skel_{\textbf{bool}}$, while

$$0.5 : \textbf{float}[B; \{0.5\}] < 0.5 : \textbf{float}[B; \{0.5\}] \quad \text{and} \quad 0.2 : \textbf{float}[B; \{0.2\}] < 0.8 : \textbf{float}[B; \{0.8\}]$$

are distinct elements of $Expr_{\texttt{bool}}$.

In particular, both $Skel_\tau$ and $Expr_\tau$ are in general uncountable, since float subtype annotations of the form $\texttt{float}[B; V]$ may vary over uncountably many real values.

Definition F.12 (Decomposing an expression). In this subsection, $Expr_\tau$ denotes the typed operational fragment whose constructors are listed in the skeleton grammar above and in Figures 26 and 27. This is the exact typed operational carrier used in this appendix, not a claim about every constructor accepted by the Slice implementation. Extending the operational semantics requires adding the corresponding skeleton and structural decomposition clauses.

The decomposition function

$$\mathrm{dec}\colon Expr_\tau \longrightarrow \bigsqcup_{s \in Skel_\tau} \left(\{s\} \times \mathbb{R}^{holes(s)} \right)$$

is defined as the inverse of filling a skeleton, i.e., for every expression $e : \tau$, structurally by replacing each float constant with a hole and collecting the removed constants in the hole-occurrence order above. Write $\mathbb{R}^0 = \{\langle\rangle\}$ for the singleton containing the empty coordinate vector and $\mathbb{R}^1 = \{\langle r \rangle \mid r \in \mathbb{R}\}$. In the recursive definition, $\vec{r}_1 \parallel \vec{r}_2$ is ordered vector concatenation, with $\langle\rangle$ as its identity; concatenation of three vectors associates to the left. A float constant contributes $\langle c \rangle$, while a constructor with no float constants contributes $\langle\rangle$. The explicit recursive definition of dec is shown in Figures 26 and 27.

Lemma F.13 (Decomposition inversion and uniqueness). *For every typed expression $e : \tau$ in the fragment above, if $\mathrm{dec}(e) = (s, \vec{r})$, then $\vec{r} \in \mathbb{R}^{holes(s)}$ and $s[\vec{r}] = e$. Conversely, if $\vec{r} \in \mathbb{R}^{holes(s)}$ and $s[\vec{r}]$ is well typed with the annotations displayed in s, then filling and decomposing yields $(s, \vec{r})$. Hence both s and the ordered coordinate vector are unique.*

Proof. By structural induction on e (and, for the converse, on s). The float constant case follows from $\mathrm{dec}(c) = (\square, \langle c \rangle)$ and $\square[\langle c \rangle] = c$. Constructors with no float constants use the unique element $\langle\rangle \in \mathbb{R}^0$. In every recursive case, the induction hypotheses reconstruct the immediate subexpressions, and the fixed left-to-right hole order splits the concatenated vector into blocks of the uniquely determined lengths $holes(s_i)$. Reapplying the same constructor reconstructs e. Discrete probability literals remain in the skeleton and therefore require no coordinate. These cases cover exactly the grammar displayed above. □

Definition F.14 (Hole assignments of a skeleton). For a given $s : \tau \in Skel_\tau$, define the set of hole assignments of s as:

$$A_{s;\tau} := \left\{ \vec{r} \in \mathbb{R}^{holes(s)} \mid s[\vec{r}] : \tau \in Expr_\tau \text{ and } \mathrm{dec}(s[\vec{r}] : \tau) = (s : \tau, \vec{r}) \right\} \subseteq \mathbb{R}^{holes(s)}$$

Intuitively, $A_{s;\tau}$ is the set of all real tuples that can be substituted into the holes of s to yield a well-typed expression of type τ.

Example F.15. Consider the skeleton

$$s : \tau = (\square : \texttt{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \texttt{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \texttt{bool}.$$

This skeleton has two holes:

- The left hole has type $\texttt{float}[\{<3.5, <4.5\}; \top]$, so it may be instantiated by any real number.
- The right hole has type $\texttt{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]$, so it may only be instantiated by one of the values 3.5 or 4.5.

An expression $e : \texttt{bool}$ has skeleton s iff it is of the form

$$e = c_1 < c_2$$

where $c_1 \in \mathbb{R}$ and $c_2 \in \{3.5, 4.5\}$.

Therefore, the admissible parameter tuples are precisely

$$A_{s:\tau} = \{(c_1, c_2) \mid c_1 \in \mathbb{R},\ c_2 \in \{3.5, 4.5\}\}.$$

$$
\mathrm{dec}(e : \tau) = \begin{cases}
(\mathrm{dec}(e_1)_1 \;\&\&\; \mathrm{dec}(e_2)_1 : \textbf{bool}, \\ \mathrm{dec}(e_1)_2 \parallel \mathrm{dec}(e_2)_2) & \text{if } e = e_1 \;\&\&\; e_2 : \textbf{bool} \\[2ex]
(\mathrm{dec}(e_1)_1 \;||\; \mathrm{dec}(e_2)_1 : \textbf{bool}, \\ \mathrm{dec}(e_1)_2 \parallel \mathrm{dec}(e_2)_2) & \text{if } e = e_1 \;||\; e_2 : \textbf{bool} \\[2ex]
(\text{not } \mathrm{dec}(e_1)_1 : \textbf{bool}, \mathrm{dec}(e_1)_2) & \text{if } e = \text{not } e_1 : \textbf{bool} \\[2ex]
((\mathrm{dec}(e_1)_1, \mathrm{dec}(e_2)_1) : \tau_1 * \tau_2, \\ \mathrm{dec}(e_1)_2 \parallel \mathrm{dec}(e_2)_2) & \text{if } e = (e_1, e_2) : \tau_1 * \tau_2 \\[2ex]
(\textbf{fst } \mathrm{dec}(e_1)_1 : \tau_1, \mathrm{dec}(e_1)_2) & \text{if } e = \textbf{fst } e_1 : \tau_1 \\[2ex]
(\textbf{snd } \mathrm{dec}(e_1)_1 : \tau_2, \mathrm{dec}(e_1)_2) & \text{if } e = \textbf{snd } e_1 : \tau_2 \\[2ex]
(\textbf{fun } x \to \mathrm{dec}(e_1)_1 : \tau_1 \to \tau_2, \mathrm{dec}(e_1)_2) & \text{if } e = \textbf{fun } x \to e_1 : \tau_1 \to \tau_2 \\[2ex]
(\textbf{fix } f\ x := \mathrm{dec}(e_1)_1 : \tau_1 \to \tau_2, \mathrm{dec}(e_1)_2) & \text{if } e = \textbf{fix } f\ x := e_1 : \tau_1 \to \tau_2 \\[2ex]
(\mathrm{dec}(e_1)_1\ \mathrm{dec}(e_2)_1 : \tau_2, \\ \mathrm{dec}(e_1)_2 \parallel \mathrm{dec}(e_2)_2) & \text{if } e = (e_1 : \tau_1 \to \tau_2\ \ e_2 : \tau_1) : \tau_2 \\[2ex]
(\mathtt{nil} : \textbf{list}(\tau_1), \langle\rangle) & \text{if } e = \mathtt{nil} : \textbf{list}(\tau_1) \\[2ex]
(\mathrm{dec}(e_1)_1 :: \mathrm{dec}(e_2)_1 : \textbf{list}(\tau_1), \\ \mathrm{dec}(e_1)_2 \parallel \mathrm{dec}(e_2)_2) & \text{if } e = e_1 :: e_2 : \textbf{list}(\tau_1) \\[2ex]
((\textbf{match } \mathrm{dec}(e_0)_1 \textbf{ with } \mid \mathtt{nil} \to \mathrm{dec}(e_{\text{nil}})_1 \\ \quad \mid h :: t \to \mathrm{dec}(e_{\text{cons}})_1 \textbf{ end}) : \tau_2, \\ \mathrm{dec}(e_0)_2 \parallel \mathrm{dec}(e_{\text{nil}})_2 \parallel \mathrm{dec}(e_{\text{cons}})_2) & \text{if } e = (\textbf{match } e_0 : \textbf{list}(\tau_1) \textbf{ with } \mid \mathtt{nil} \to e_{\text{nil}} : \tau_2 \\ & \qquad\qquad \mid h :: t \to e_{\text{cons}} : \tau_2 \textbf{ end}) : \tau_2
\end{cases}
$$

Fig. 27. Recursive definition of dec over typed expressions (continued).

Definition F.18 (Expressions of a skeleton). For a given $s : \tau \in \mathit{Skel}_\tau$, define the set of well-typed expressions having skeleton s as:

$$\mathit{Expr}_{s;\tau} := \left\{ e : \tau \in \mathit{Expr}_\tau \;\middle|\; \mathrm{dec}(e : \tau)_1 = s : \tau \text{ and } \mathrm{dec}(e : \tau)_2 \in A_{s;\tau} \right\} \subseteq \mathit{Expr}_\tau.$$

Intuitively, $\mathit{Expr}_{s;\tau}$ consists exactly of those expressions whose syntactic shape is fixed by s, and whose real constants vary according to the admissible hole assignments in $A_{s;\tau}$.

Example F.19. Consider the skeleton

$$s : \tau = (\square : \textbf{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \textbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \textbf{bool}.$$

An expression $e : \textbf{bool}$ belongs to $\mathit{Expr}_{s;\tau}$ iff it is of the form

$$e = c_1 < c_2$$

where $c_1 \in \mathbb{R}$ and $c_2 \in \{3.5, 4.5\}$.

Therefore,

$$\mathit{Expr}_{s;\tau} = \{c_1 < c_2 \mid c_1 \in \mathbb{R},\ c_2 \in \{3.5, 4.5\}\}.$$

Definition F.20 (σ-algebra construction on $Expr_{s;\tau}$). Let $s : \tau \in Skel_\tau$. Endow $Expr_{s;\tau}$ with the following σ-algebra:

$$\mathcal{F}_{Expr_{s;\tau}} := \left\{ \{e : \tau \in Expr_{s;\tau} \mid \mathrm{dec}(e : \tau)_2 \in B\} \mid B \in \mathcal{F}_{s;\tau} \right\}.$$

Lemma F.21. *For a fixed $s : \tau \in Skel_\tau$, the map*

$$\mathrm{dec}_{s;\tau} : Expr_{s;\tau} \to A_{s;\tau}, \qquad e \mapsto \mathrm{dec}(e)_2.$$

is a bijection.

Proof. Let $s : \tau \in Skel_\tau$.

(i) Injectivity: Suppose $e_1, e_2 \in Expr_{s;\tau}$ satisfy $\mathrm{dec}_{s;\tau}(e_1) = \mathrm{dec}_{s;\tau}(e_2)$. By definition of $\mathrm{dec}_{s;\tau}$, $\mathrm{dec}_{s;\tau}(e_1) = \mathrm{dec}_{s;\tau}(e_2) = \vec{r}$ for some $\vec{r} \in A_{s;\tau}$. Since both e_1 and e_2 have skeleton s by assumption, we have $e_1 = s[\vec{r}]$ and $e_2 = s[\vec{r}]$, hence $e_1 = e_2$.

(ii) Surjectivity: Let $\vec{r} \in A_{s;\tau}$. By definition of $A_{s;\tau}$, there exists some expression $e \in Expr_\tau$ such that $\mathrm{dec}(e) = (s, \vec{r})$. Then $e = s[\vec{r}]$ is in $Expr_\tau$ and $\mathrm{dec}(e) = (s, \vec{r})$. Therefore $e \in Expr_{s;\tau}$ by construction of $Expr_{s;\tau}$, and $\mathrm{dec}_{s;\tau}(e) = \vec{r}$.

$\mathrm{dec}_{s;\tau}$ is injective and surjective, hence a bijection. □

Lemma F.22 (Isomorphism property). *For $s \in Skel_\tau$,*

$$(Expr_{s;\tau}, \mathcal{F}_{Expr_{s;\tau}}) \cong (A_{s;\tau}, \mathcal{F}_{s;\tau}).$$

Proof. We know $(A_{s;\tau}, \mathcal{F}_{s;\tau})$ is a Borel measurable space by Lemma F.17. First, verify that $\mathcal{F}_{Expr_{s;\tau}}$ coincides with the σ-algebra generated by $\mathrm{dec}_{s;\tau}$, i.e. $\mathcal{F}_{Expr_{s;\tau}} = \mathrm{dec}_{s;\tau}^{-1}(\mathcal{F}_{s;\tau})$:

$$\begin{aligned}
\mathcal{F}_{Expr_{s;\tau}} &:= \left\{ \{e : \tau \in Expr_{s;\tau} \mid \mathrm{dec}(e : \tau)_2 \in B\} \mid B \in \mathcal{F}_{s;\tau} \right\} && \text{(Definition F.20)} \\
&:= \left\{ \mathrm{dec}_{s;\tau}^{-1}(B) \mid B \in \mathcal{F}_{s;\tau} \right\} && \text{(unfolding the preimage of } \mathrm{dec}_{s;\tau}) \\
&:= \mathrm{dec}_{s;\tau}^{-1}(\mathcal{F}_{s;\tau}) && \text{(Definition F.2)}
\end{aligned}$$

By Lemma F.21, $\mathrm{dec}_{s;\tau} : Expr_{s;\tau} \to A_{s;\tau}$ is a bijection. By Lemma F.3, $\mathrm{dec}_{s;\tau}$ induces an isomorphism $(Expr_{s;\tau}, \mathcal{F}_{Expr_{s;\tau}}) \cong (A_{s;\tau}, \mathcal{F}_{s;\tau})$. □

Lemma F.23. *$(Expr_{s;\tau}, \mathcal{F}_{Expr_{s;\tau}})$ is a Borel measurable space.*

Proof. By Lemma F.22. □

Now we combine the measurable spaces for each distinct skeleton into a single measurable space of all well-typed expressions of type τ. The set of all well-typed expressions of type τ is thus the disjoint union:

$$Expr_\tau = \bigsqcup_{s:\tau \in Skel_\tau} Expr_{s;\tau}.$$

This union is disjoint because every typed expression $e : \tau$ has a unique skeleton, namely $\mathrm{dec}(e)_1$, so e belongs to exactly one set $Expr_{s;\tau}$.

Definition F.24 (σ-algebra construction on $Expr_\tau$). Endow $Expr_\tau$ with the following σ-algebra:

$$\mathcal{F}_{Expr_\tau} := \{A \subseteq Expr_\tau \mid \forall s \in Skel_\tau.\ A \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}\}$$

Lemma F.25. *$(Expr_\tau, \mathcal{F}_{Expr_\tau})$ is a measurable space.*

Proof. We verify that $\mathcal{F}_{Expr_\tau}$ is indeed a σ-algebra on $Expr_\tau$.